Du cadrage à l'interface
La démarche complète : TP1, TP2, TP3

A. Larhlimi

Développement et déploiement d'applications intelligentes
Trois séances de 2 h · travail en binôme · 12 octobre 2026

Table des matières

1 Du cadrage à l'interface : la démarche complète, TP1, TP2, TP3 **1**
1.1 Vue d'ensemble . . . 2
1.1.1 Le fil rouge . . . 2
1.1.2 Les trois kits . . . 2
1.2 Avant la première séance . . . 2

2 TP1 : cadrer et préparer l'environnement **2**
2.1 Étape 1. Décompresser le kit (5 min, chacun) . . . 3
2.2 Étape 2. L'environnement virtuel (10 min, chacun) . . . 3
2.3 Étape 3. Les trois fiches de cadrage (50 min, à deux) . . . 3
2.4 Étape 4. Le dépôt GitHub (25 min, membre A) . . . 3
2.5 Étape 5. L'installation complète (15 min, les deux) . . . 4
2.6 Étape 6. Un commit par membre (10 min, chacun) . . . 4
2.7 Étape 7 et 8. Vérification et clone frais (15 min) . . . 4

3 TP2 : entraîner, évaluer, sauvegarder le modèle **4**
3.1 Étape A. Installer le kit (15 min) . . . 4
3.2 Étape B. Le jeu de démonstration (15 min) . . . 5
3.3 Étape C. Entraîner (25 min) . . . 5
3.4 Étape D. Évaluer et comprendre (30 min) . . . 5
3.5 Étape E. Le vrai jeu et les tests (35 min) . . . 5

4 TP3 : l'interface de démonstration **6**
4.1 Étape A. Installer et lancer (15 min) . . . 6
4.2 Étape B. L'onglet « un avis » (30 min) . . . 6
4.3 Étape C. L'onglet « un fichier CSV » (40 min) . . . 6
4.4 Étape D. Tests, documentation, push (35 min) . . . 6
4.5 Variante Google Colab . . . 7
4.6 Le compte rendu global . . . 7
4.7 Récapitulatif des treize contrôles . . . 7
4.8 Les erreurs les plus coûteuses, tous TP confondus . . . 8
4.9 La structure finale du dépôt . . . 8

1 Du cadrage à l'interface : la démarche complète, TP1, TP2, TP3

Module : Développement et déploiement d'applications intelligentes **Auteur :** A. Larhlimi **Public :** binômes d'étudiants · **Durée totale :** trois séances de 2 h

Ce document enchaîne les trois premières séances sur un seul et même dépôt. À la fin, vous avez une application de sentiment complète : un modèle entraîné, testé, et une interface qu'une personne non technique peut utiliser.

1.1 Vue d'ensemble

Séance	Ce que vous produisez	Contrôles	Tests cumulés
TP1	Dépôt structuré, environnement reproductible, trois fiches de cadrage	T0 à T7	38
TP2	Le modèle : entraînement, évaluation, métriques versionnées	C1 à C5	42
TP3	L'interface de démonstration Streamlit	S1 à S4	45

Un seul dépôt, `sentiment-app`, du premier jour au dernier. Chaque kit se pose sur le précédent sans rien écraser.

1.1.1 Le fil rouge

Une équipe relation client reçoit des milliers d'avis par jour et les traite dans l'ordre d'arrivée. On veut faire remonter les avis négatifs en tête de file. Problème volontairement simple côté modèle, pour que la difficulté soit là où elle est vraiment : la reproductibilité, les tests, le passage à l'utilisateur.

1.1.2 Les trois kits

Kit	Contenu	Se pose sur
`sentiment-app.zip`	Le dépôt de départ complet : structure, scripts, 38 tests, CI	rien, c'est la base
`tp2-semaine2-kit.zip`	`src/` (quatre scripts), `tests/test_model.py`	le dépôt du TP1
`tp3-streamlit-kit.zip`	`ui/` (interface et helpers), `tests/test_ui.py`	le dépôt après TP2

Les deux derniers s'installent par un script : `python appliquer_tp2.py ../sentiment-app`.

1.2 Avant la première séance

Sur chaque poste : **Python 3.11** (3.10 toléré, évitez 3.12 et 3.13), **Git**, un éditeur, un **compte GitHub**.

```
python --version                    # 3.11.x attendu ; sinon py -3.11 --version
git --version

git config --global user.name "Prénom Nom"        # compté par le contrôle T5
git config --global user.email "prenom.nom@ecole.ma"
git config --global init.defaultBranch main
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned   # autorise l'activation du
    venv
```

Répartissez les rôles : le **membre A** crée le dépôt et pousse le kit ; le **membre B** clone. Tout le reste se fait à deux.

---

2 TP1 : cadrer et préparer l'environnement

Objectif : *décider quelle approche technique employer pour trois besoins métier, et monter un dépôt que l'enseignant peut cloner et faire tourner en trois commandes.*

2.1 Étape 1. Décompresser le kit (5 min, chacun)

```
unzip sentiment-app.zip        # Windows : clic droit, Extraire tout
cd sentiment-app               # toutes les commandes partent d'ici
ls ; ls -Force | findstr gitignore
```

Contrôle T0 : les sept dossiers (`app`, `data`, `docs`, `models`, `scripts`, `tests`, `ui`), les documents, et `.gitignore`, qui ne s'affiche pas avec un `ls` ordinaire. C'est le seul rempart contre l'erreur de T3.

2.2 Étape 2. L'environnement virtuel (10 min, chacun)

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1          # Linux, macOS : source .venv/bin/activate
python -m pip install pandas scikit-learn pytest
python -m pytest -q
```

Contrôle T1 : `3 failed, 31 passed, 4 skipped.` Ce rouge est voulu : les trois échecs sont les fiches vides, les quatre tests ignorés attendent le dépôt Git. Si vous obtenez `38 passed` ici, vous testez un autre dossier.

Votre invite doit afficher `(.venv)`. L'activation ne vaut que pour la fenêtre courante.

2.3 Étape 3. Les trois fiches de cadrage (50 min, à deux)

Lisez d'abord `docs/cadrage/EXEMPLE-corrige.md`. La grille a quatre critères : (a) données étiquetées, (b) vérifiabilité, (c) coût par requête, (d) conséquence d'une erreur.

L'arbre de choix, dans l'ordre : logique connue et stable ? **règles métier**. Données étiquetées ? **machine learning**. Réponse à tirer de documents et à citer ? **RAG**. Sinon seulement, **génératif seul**.

Fiche	Le cas	Réponse attendue
`cas1-pharmacie.md`	Détecter les ordonnances incomplètes	Règles métier : (a) pas d'étiquettes, (b) explicabilité, (d) erreur grave
`cas2-reglement.md`	Répondre sur le règlement intérieur	RAG : (b) citer l'article, (a) pas d'étiquettes, (c) volume faible
`cas3-avis-negatifs.md`	Prioriser les avis négatifs	Machine learning : (a) étiquettes gratuites, (c) volume, (d) erreur peu grave

Trois règles vérifiées par les tests : une seule case `- [x]` par fiche ; au moins deux critères cités **par leur lettre** ; chaque rubrique obligatoire d'au moins une phrase.

```
python -m pytest tests\test_cadrage.py -q
```

Contrôle T2 : `12 passed.` Les messages d'échec nomment la fiche et la rubrique fautives.

2.4 Étape 4. Le dépôt GitHub (25 min, membre A)

Sur github.com : New repository, nom `sentiment-app`, **Private**, aucune case cochée. Settings, Collaborators : membre B en **Write**, enseignant en **Read**.

```
git init
git add -A
git status
```

Contrôle T3 : aucune ligne `.venv/` dans la sortie. Si elle apparaît, arrêtez-vous : le `.gitignore` manque. Une fois poussé, le dépôt pèse 300 Mo et le test reste rouge même après correction, car il lit tout l'historique.

```
git commit -m "Semaine 1 : structure du depot et kit TP1"
git branch -M main
git remote add origin https://github.com/votre-compte/sentiment-app.git
git remote -v                       # vérifiez : pas de chevrons dans l'URL
git push -u origin main
```

Au premier push, le mot de passe du compte est refusé : il faut un **jeton d'accès personnel** (Settings, Developer settings, Personal access tokens, portée `repo`). Raccourci : `bash scripts/init_depot.sh <url>`, qui refuse de pousser si `.venv/` est en attente.

2.5 Étape 5. L'installation complète (15 min, les deux)

Le membre B accepte l'invitation reçue par courriel, puis clone.

```
git clone https://github.com/votre-compte/sentiment-app.git
cd sentiment-app
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1   # Linux, macOS : bash
    scripts/setup.sh
python scripts\check_setup.py
```

Contrôle T4 : `0 erreur(s)`. Comptez 5 à 10 minutes : `requirements.txt` installe tout le semestre. L'avertissement « 1 auteur seulement » est normal ici.

`requirements.lock` est régénéré sur chaque poste : **un seul membre le commite**.

2.6 Étape 6. Un commit par membre (10 min, chacun)

```
git pull
# ajoutez votre ligne au tableau du binôme dans README.md
git add README.md
git commit -m "README : ajout de <nom> au tableau du binome"
git push
git shortlog -sn --all
python -m pytest tests\test_git_history.py -q
```

Contrôle T5 : deux noms distincts, puis `3 passed`. Si le test ne voit qu'un auteur, les deux postes partagent le même `user.name`.

2.7 Étape 7 et 8. Vérification et clone frais (15 min)

```
git pull
python scripts\check_setup.py      # T6 : 0 erreur, 0 avertissement
python -m pytest -q                # T6 : 38 passed
```

Puis l'onglet **Actions** doit être vert. Enfin, le test de l'enseignant :

```
bash scripts/verif_clone.sh https://github.com/votre-compte/sentiment-app.git
```

Contrôle T7 : `0 erreur(s)` et `38 passed` sur un clone frais, sans aucune intervention manuelle.

---

3 TP2 : entraîner, évaluer, sauvegarder le modèle

> ***Objectif :*** *produire un modèle reproductible, évalué correctement, et sauvegardé dans un fichier que l'interface du TP3 pourra charger. Pas un bon modèle : un modèle dont vous savez pourquoi il se trompe.*

3.1 Étape A. Installer le kit (15 min)

Décompressez `tp2-semaine2-kit.zip` **à côté** du dépôt, pas dedans.

```
cd tp2-semaine2
python appliquer_tp2.py ../sentiment-app --simulation
python appliquer_tp2.py ../sentiment-app
cd ../sentiment-app && source .venv/bin/activate
python -m pytest -q
```

Contrôle C1 : `2 failed, 38 passed, 2 skipped`. Les 38 verts sont ceux du TP1. Les 2 échecs réclament `models/metrics.json`, les 2 ignorés attendent le modèle.

3.2 Étape B. Le jeu de démonstration (15 min)

```
python src/make_demo.py
```

Contrôle C2 : 290 avis, 145 de chaque classe. Le fichier contient 80 avis nettement positifs, 80 nettement négatifs, 100 **mitigés** et 30 neutres.

Lisez trois avis mitigés : une première moitié d'un sentiment, un retournement (« mais finalement »), une seconde moitié de l'autre. L'étiquette suit la seconde, celle qui conclut. Cette structure explique toutes les erreurs de l'étape D.

3.3 Étape C. Entraîner (25 min)

```
python src/train.py
```

Contrôle C3 : F1 macro 0,879, et deux fichiers produits.

```
Source : demo, 290 avis (145 négatifs)
232 avis d'entraînement, 58 de test
Vocabulaire : 316 termes
F1 macro sur le jeu de test : 0.879 (seuil 0.85)
```

```
git status --short
```

`models/metrics.json` apparaît, `models/sentiment.joblib` non, `data/demo_reviews.csv` non plus. **Ce qui se régénère par une commande n'entre pas dans Git ; seule la mesure est versionnée.**

Trois lignes de code à lire avant de continuer : le `train_test_split` **avant** toute vectorisation (sinon le jeu de test fuite dans l'entraînement), le `Pipeline` qui enferme vectoriseur et modèle dans un seul objet, et `random_state=42` qui rend le découpage reproductible.

3.4 Étape D. Évaluer et comprendre (30 min)

```
python src/evaluate.py
```

Contrôle C4 : 7 erreurs sur 58.

```
            negative  positive
  negative        27         2
  positive         5        24
```

Les deux faux positifs sont le cas qui compte : un client mécontent traité en retard. Toutes les erreurs portent sur des avis mitigés : le modèle compte des mots positifs et négatifs puis tranche, mais ne voit pas le retournement, parce qu'un sac de mots **n'a pas d'ordre**.

Écrivez `docs/semaine2.md`, trois phrases : quelles erreurs, pourquoi, et ce qu'il faudrait pour les corriger.

3.5 Étape E. Le vrai jeu et les tests (35 min)

```
python src/train.py --allocine     # 20 000 avis publics, 7,6 Mo
python -m pytest -q
git add -A && git commit -m "Semaine 2 : modele de sentiment, metriques et tests"
    && git push
```

Contrôle C5 : F1 macro 0,920, puis `42 passed`, puis CI verte.

	Démonstration	Allociné
Avis	290	20 000
Vocabulaire	316 termes	149 529 termes
F1 macro	0,879	0,920

Le modèle n'a pas changé : ce sont les données, et la mesure devient enfin crédible, calculée sur 4 000 avis au lieu de 58.

La CI exécute les 42 tests sans jamais avoir le modèle : les deux tests de prédiction sont ignorés, les deux tests de qualité lisent `metrics.json`, qui est versionné. Votre intégration continue vérifie qu'un modèle acceptable a été produit, **sans transporter le modèle**.

---

4 TP3 : l'interface de démonstration

> ***Objectif :*** *rendre le modèle utilisable par quelqu'un qui n'ouvrira jamais un terminal, sans réécrire une ligne de logique de prédiction.*

4.1 Étape A. Installer et lancer (15 min)

```
cd tp3-streamlit
python appliquer_tp3.py ../sentiment-app
cd ../sentiment-app && source .venv/bin/activate
streamlit run ui/app.py
```

Contrôle S1 : le navigateur s'ouvre sur `http://localhost:8501`, trois onglets s'affichent. Si le modèle manque, la page le dit et s'arrête : lancez `python src/train.py -allocine`.

Ce terminal reste occupé tant que l'interface tourne. Ouvrez-en un second pour la suite, et arrêtez avec `Ctrl+C`.

4.2 Étape B. L'onglet « un avis » (30 min)

Saisissez un avis clairement positif, un clairement négatif, puis un mitigé : « service impeccable au début, mais finalement produit cassé ».

Contrôle S2 : l'avis mitigé affiche « à vérifier », pas un verdict tranché, dès que la confiance tombe sous 60 %.

Le seuil est dans `ui/helpers.py`, constante `SEUIL_INCERTITUDE`. Faites-le varier à 0,50 puis 0,75 : plus il est haut, plus le modèle renvoie de cas à un agent, et moins il se trompe tout seul. C'est un arbitrage métier, pas un réglage technique, et il découle directement de la fiche de cadrage du cas 3.

4.3 Étape C. L'onglet « un fichier CSV » (40 min)

Téléversez `data/sample_reviews.csv`, puis `data/demo_reviews.csv`.

Contrôle S3 : trois métriques (avis analysés, négatifs détectés et leur pourcentage, cas à vérifier), un graphique de répartition, un tableau trié négatifs en tête, et un bouton de téléchargement.

Comparez la part de négatifs annoncée avec le vrai étiquetage du fichier de démonstration : l'écart, ce sont les erreurs mesurées au TP2.

4.4 Étape D. Tests, documentation, push (35 min)

```
python -m pytest tests/test_ui.py -q      # 3 passed
python -m pytest -q                       # 45 passed
git add -A && git commit -m "Semaine 3 : interface de demonstration Streamlit" &&
    git push
```

Contrôle S4 : `45 passed` et CI verte.

Les tests d'interface **ne lancent pas Streamlit** : ils importent `ui/helpers.py`. C'est pour cela que la logique est séparée de l'affichage, et c'est ce que vérifie `test_helpers_importables`. Une interface non testable est une interface qu'on casse sans s'en apercevoir.

Écrivez `docs/semaine3.md`, trois phrases : quel seuil d'incertitude vous retenez, combien d'avis cela envoie à un agent, et pourquoi ce compromis vous paraît juste.

4.5 Variante Google Colab

Les trois TP ont leur notebook, dans `notebooks/` : `TP1-colab.ipynb`, `TP2-colab.ipynb` et `TP3-colab-streamlit.ipynb`. Mêmes étapes, mêmes contrôles, avec trois différences.

Pas d'environnement virtuel. Colab installe dans son Python système, donc `test_virtualenv_active` est écarté : les comptes deviennent 37, 41 et 44 au lieu de 38, 42 et 45.

Streamlit a besoin d'un tunnel. Le serveur tourne sur la machine virtuelle de Colab, que votre navigateur ne peut pas joindre. Le notebook du TP3 lance `localtunnel`, affiche un mot de passe (l'adresse IP publique de la machine) et une adresse `loca.lt` à ouvrir. Cette adresse est publique et temporaire : n'y laissez jamais passer de données réelles.

Rien ne survit à la session. `/content` disparaît à la fermeture. Poussez sur GitHub avant de partir. Le modèle, lui, n'a pas à être conservé : il se réentraîne par une commande.

4.6 Le compte rendu global

À la fin des trois séances, chaque binôme rédige un compte rendu unique, déposé dans son dépôt à l'emplacement `docs/compte-rendu.md`, puis figé par une étiquette Git :

```
cp docs/compte-rendu-TEMPLATE.md docs/compte-rendu.md
# rédaction...
git add docs/compte-rendu.md
git commit -m "Compte rendu global des TP1 a TP3"
git push
git tag -a compte-rendu -m "Version remise le <date>" && git push origin compte-
    rendu
```

Huit sections, quatre à six pages, noté sur 20 : cadrage, architecture, modèle, analyse des erreurs, seuil d'incertitude, ce qui a échoué, ce que vous feriez autrement, répartition du travail. L'énoncé complet et la grille sont dans `ENONCE-COMPTE-RENDU.md`, le squelette à remplir dans `docs/compte-rendu-TEMPLATE.md`.

Deux règles qui font la différence : citez **vos** chiffres, ceux de votre `models/metrics.json`, et tirez au moins un exemple d'erreur de vos propres données. Un compte rendu sans aucun échec mentionné n'est pas crédible.

4.7 Récapitulatif des treize contrôles

	Commande	**Résultat attendu**
T0	`ls` et `ls -a`	7 dossiers, `.gitignore` présent
T1	`pytest -q`	3 failed, 31 passed, 4 skipped
T2	`pytest tests/test_cadrage.py -q`	12 passed
T3	`git status` avant le commit	aucune ligne `.venv/`
T4	`python scripts/check_setup.py`	0 erreur(s)
T5	`git shortlog -sn -all` puis `pytest tests/test_git_history.py`	deux noms, 3 passed
T6	`check_setup.py` et `pytest -q`	0 erreur, 0 avertissement, 38 passed
T7	la suite sur un clone frais	38 passed sans intervention
C1	`pytest -q`	2 failed, 38 passed, 2 skipped
C2	`python src/make_demo.py`	290 avis, 145 de chaque classe
C3	`python src/train.py`	F1 macro 0,879, `metrics.json` seul en attente

	Commande	Résultat attendu
C4	`python src/evaluate.py`	7 erreurs sur 58, toutes sur des avis mitigés
C5	`train.py -allocine` puis `pytest -q`	F1 0,920, 42 passed, CI verte
S1	`streamlit run ui/app.py`	la page s'ouvre, trois onglets
S2	un avis mitigé	verdict « à vérifier » sous 60 %
S3	un CSV dans l'onglet 2	métriques, graphique, tableau trié, export
S4	`pytest -q`	45 passed, CI verte

4.8 Les erreurs les plus coûteuses, tous TP confondus

Message	Cause	Correction
`pytest n'est pas reconnu`	Venv non activé	Activer, ou `python -m pytest`
`Activate.ps1 cannot be loaded`	Politique PowerShell	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`ModuleNotFoundError`	Venv non activé, ou installation partielle	`where python` doit contenir `.venv`
`error: 400` sur l'URL	Chevrons restés dans l'adresse	`git remote set-url origin <vraie URL>`
`Password authentication is not supported`	Authentification	Jeton d'accès personnel
`403 Forbidden`	Autorisation	Accepter l'invitation de collaborateur
`.venv/ a ÃľtÃľ commitÃľ`	`.gitignore` absent au premier `add`	Refaire le dépôt (voir `DEPANNAGE.md`)
`models/metrics.json absent`	Étape C du TP2 sautée	`python src/train.py`
`models/sentiment.joblib absent`	Modèle non entraîné sur ce poste	`python src/train.py -allocine`
`Port 8501 is already in use`	Une interface tourne déjà	`Ctrl+C`, ou `-server.port 8502`

4.9 La structure finale du dépôt

```
sentiment-app/
|-- README.md, TP1-ENONCE.md, MARCHE-A-SUIVRE.md, TESTS.md, AIDE-GIT.md, DEPANNAGE.
    md
|-- requirements.txt, requirements.lock, .gitignore, .env.example, pytest.ini,
    Makefile
|-- .github/workflows/ci.yml        intégration continue
|-- docs/
|   |-- cadrage/cas1, cas2, cas3    TP1, remplies (livrable noté)
|   |-- semaine2.md                 TP2, analyse des erreurs
|   |-- semaine3.md                 TP3, seuil d'incertitude
|-- src/      make_demo, train, evaluate, predict        (TP2)
|-- ui/       app.py, helpers.py                         (TP3)
|-- tests/    environment (23), cadrage (12), git (3), model (4), ui (3)
|-- scripts/  check_setup, setup, init_depot, verif_clone, new_cadrage
|-- models/   .gitkeep + metrics.json      le modèle lui-même n'est JAMAIS commité
|-- data/     sample_reviews.csv           demo_reviews.csv est régénérable
```

45 tests à l'arrivée. Trois artefacts restent hors de Git parce qu'ils se reconstruisent en une commande : le venv, le jeu de démonstration, le modèle. Une seule mesure est versionnée : `models/metrics.json`. C'est le principe qui portera tout le reste du semestre.